PHASE 3 INSTRUCTIONS (FRONT-END)

To use: HTML/CSS/JS

Provided below are the file names, their essential instructions (no design, only requirements IN ORDER TO design), and their respective directory locations in which you will create in VSCode (recommended IDE).

-- FILE NAME: -- templates/index.html

- **LOCATION:** app/templates/index.html
- **What it does / content instructions:**
  - Acts as the single-page entry for users (Home / Product listing). Should include:
    - A <header> with site title and navigation links: Home, Catalog, Cart (show item count), Login/Register (or Logout when authenticated), and a conditional Admin link visible only if the logged-in user has role admin.
    - A search input (id="searchInput") and a filter area: min price (id="minPrice"), max price (id="maxPrice"), active checkbox (id="onlyActive"), and a submit button (id="searchBtn").
    - A product list container (id="productList") where JS will append product cards. Each product card must include:
      - SKU (class="product-sku"), name (class="product-name"), description (class="product-desc"), price (class="product-price"), quantity (class="product-qty"), average rating (class="product-rating"), and a "View" button (class="product-view-btn" data-product-id="...").
    - Pagination controls: previous (id="prevPage"), next (id="nextPage"), and a span showing current page (id="pageNum").
    - Footer with brief contact or copyright.
  - Include these <script> tags at bottom:
    - <script src="/static/js/main.js" defer></script>
    - <link rel="stylesheet" href="/static/css/main.css">
  - Accessibility: product view buttons should be <button> elements.

-- FILE NAME: -- templates/product.html

- **LOCATION:** app/templates/product.html
- **What it does / content instructions:**
  - Product detail page. Must include:
    - Container for product images (id="productImages"). If multiple images, allow carousel behavior (prev/next buttons).
    - Name (id="productName"), SKU (id="productSKU"), full description (id="productDesc"), price (id="productPrice"), and available stock (id="productStock").
    - Add-to-cart form:
      - Quantity input (id="addQty", number).
      - "Add to cart" button (id="addToCartBtn").
    - Reviews section (id="reviewsList") — each review item must include reviewer name (class="rev-user"), rating (class="rev-rating"), date (class="rev-date"), body (class="rev-body"), and replies nested inside (class="rev-replies").
    - Review submission form (visible for authenticated customers): rating select (id="reviewRating"), title (id="reviewTitle"), body (id="reviewBody"), submit (id="submitReview").
    - For Sales users: each review shows a "Reply" button (class="reply-btn" data-review-id="...") and a reply form appears inline when clicked.
    - For Admin users: each review shows a "Hide" button (class="hide-review-btn" data-review-id="...").
  - Include links to static/js/product.js and static/css/product.css.

-- FILE NAME: -- templates/cart.html

- **LOCATION:** app/templates/cart.html
- **What it does / content instructions:**
  - Persistent cart interface for logged-in users. Must display:
    - List of cart items (id="cartItems"). Each item displays product name, unit price, quantity with an editable input (class="cart-qty" data-item-id="..."), subtotal (class="cart-subtotal"), and a "Remove" button (class="remove-item-btn" data-item-id="...").
    - Totals area: subtotal (id="cartSubtotal"), taxes (placeholder id="cartTaxes"), grand total (id="cartTotal").
    - Checkout button (id="checkoutBtn").
  - On quantity change, frontend updates subtotal locally and sends PATCH/POST to /api/cart/items or re-sends add with new quantity (choose consistent approach). Use credentials: "include" on fetch.
  - After successful checkout, redirect to /orders.html or show confirmation.

-- FILE NAME: -- templates/orders.html

- **LOCATION:** app/templates/orders.html
- **What it does / content instructions:**
  - Shows logged-in user's order history. Must include:
    - Orders list container (id="ordersList"). For each order show order number, date, status, total price, and clickable expand to show order items.
    - Each order item shows product name, quantity, unit price, and subtotal.
  - Fetch from GET /api/orders (use credentials include). If no orders, show friendly message.

-- FILE NAME: -- templates/admin/dashboard.html

- **LOCATION:** app/templates/admin/dashboard.html
- **What it does / content instructions:**
  - Admin dashboard (only visible to admin role). Must include these panels:
    - **Users**: quick list (id=adminUsersList) and ability to search user by email; ability to toggle is_active (enable/disable user).
    - **Inventory Audit**: table (id=inventoryAuditTable) showing recent inventory changes (product SKU, changed_by, delta, reason, datetime) fetch from a new endpoint you can add later like /api/admin/inventory-audits
    - **Product Management**: quick create product form pointing to POST /api/products (multipart form). Each product row has Edit and Delete buttons; Edit opens a modal/form to PUT /api/products/<id>.
    - **Review Moderation**: list of recent reviews (id=moderationReviewsList) with Hide button which calls POST /api/reviews/<id>/hide.
  - Admin UI must always include logout action (POST to /auth/logout with credentials include).

-- FILE NAME: -- templates/sales/dashboard.html

- **LOCATION:** app/templates/sales/dashboard.html
- **What it does / content instructions:**
  - Sales dashboard with:
    - List of reviews that need a reply (id="salesReviewsQueue"). Each review has Reply button that posts to POST /api/reviews/<id>/reply with JSON {"body":"...reply text..."}.
    - Quick search for customer orders (optional).
  - Keep UI simple: list, reply form, submit button.

-- FILE NAME: -- static/css/main.css

- **LOCATION:** app/static/css/main.css
- **What it does / content instructions:**
  - Global site styles: reset or minimal baseline, header/navigation, container widths, product card layout, responsive grid for product listing, button styles, form styles.
  - Class/ID guidance (so JS can target reliably):
    - .product-card for product cards
    - .btn for buttons
    - .hidden utility class to hide elements
    - .flex-row, .flex-col helpers for layout
  - Keep CSS sleek and simple (may or may not use Tailwind); keep it modular and well-commented.

-- FILE NAME: -- static/css/product.css

- **LOCATION:** app/static/css/product.css
- **What it does / content instructions:**
    - Styles specific to the product detail page: image carousel, review list, forms spacing, reply box styling.

-- FILE NAME: -- static/js/main.js

- **LOCATION:** app/static/js/main.js
- **What it does / content instructions:**
  - Main site behavior and product listing logic:
    - On page load of index.html, fetch /api/products?page=1&per_page=10 and render into #productList.
    - Wire the search/filter inputs to call /api/products with query params (q, min_price, max_price, active, page, per_page).
    - Attach click handlers to .product-view-btn that redirect to product.html?product_id=<id>.
    - Handle pagination: clicking #nextPage increments page and fetches next page; #prevPage decrements.
    - Show a spinner or loading indicator while network calls are in progress.
  - All fetch requests that require authentication must set credentials: 'include'.
  - On responses, handle HTTP 401/403 gracefully: show a login prompt or redirect to login page.

-- FILE NAME: -- static/js/product.js

- **LOCATION:** app/static/js/product.js
- **What it does / content instructions:**
    - Fetch product details from /api/products or a dedicated endpoint such as /api/products/<id>. If the backend does not expose a single-product endpoint, fetch list and filter by id.
    - Render product images and attach next/prev behavior for the image container.
    - Implement Add-to-Cart: POST to /api/cart/items with JSON {"product_id":<id>,"quantity":<qty>} and credentials: 'include'. On success, update Cart count in header.
    - Load and render reviews (already returned with product listing) and provide submission form to POST /api/products/<id>/reviews with JSON {"rating":<1-5>,"title":"...","body":"..."}.
    - For Sales role replies: open inline reply form, POST {"body":"..."} to /api/reviews/<id>/reply.
    - For Admin hide: POST /api/reviews/<id>/hide.

-- FILE NAME: -- static/js/auth.js

- **LOCATION:** app/static/js/auth.js
- **What it does / content instructions:**
  - Handles login and register forms (pages or modal). Requirements:
    - **Login flow**: POST /auth/login with JSON {"email":"...","password":"..."}. Use credentials: 'include'. On success, backend returns {"ok":true,"role":"<role>"} — use role to update UI (show admin link if admin). On failure show message from response.
    - **Register flow**: POST /auth/register with JSON {"email":"...","password":"...","full_name":"..."}. Expect HTTP 201 on success. After register, either auto-login or prompt to login.
    - **Logout**: POST /auth/logout (no body) with credentials: 'include'. On success, clear UI state and redirect to home.
  - CSRF: because the backend uses Flask-WTF CSRF, for forms served by Flask templates you can include CSRF token in a hidden input. For pure JS/JSON API flows, the backend CSRF protection is configured to allow XHR JSON if CSRF token is not required (current setup uses CSRFProtect() if that blocks JSON POSTs you will need to fetch the CSRF token as a cookie or endpoint and include it in request headers "X-CSRFToken": "<token>" backend changes may be needed). If you prefer to avoid CSRF complexity for API-only POSTs, you can set the POST endpoints to skip CSRF for JSON API calls.
  - After login, change header to show username and role, and show Logout button.

-- FILE NAME: -- static/js/cart.js

- **LOCATION:** app/static/js/cart.js
- **What it does / content instructions:**
    - Fetch GET /api/cart and render items.
    - For quantity edits, send POST /api/cart/items with new total quantity or implement a dedicated update endpoint (coordinate with backend team which pattern to use). When removing item, send DELETE /api/cart/items/<item_id>.
    - On checkout (#checkoutBtn), POST /api/checkout — handle insufficient stock responses (display a clear message).

-- FILE NAME: -- static/js/admin.js

- **LOCATION:** app/static/js/admin.js
- **What it does / content instructions:**
    - Admin interactions: create/ edit/ delete products via the /api/products endpoints. POST for creation uses FormData with sku, name, price, quantity, description, and optionally an image file. Example: on create form submit, construct FormData, append fields, include image file input as image, then fetch('/api/products', { method:'POST', body: formData, credentials:'include' }).
    - Show inventory audit table: call endpoint that returns inventory_audits (if not available in API, coordinate with backend to add GET /api/admin/inventory-audits) and render rows.
    - User management: fetch user list and toggle is_active via a PUT or a dedicated endpoint; coordinate with backend if needed.

-- FILE NAME: -- static/img/placeholder.png (DO NOT CREATE THIS FILE MANUALLY! THIS FILE IS ONLY AN IMAGE OF YOUR CHOICE THAT ACTS AS A PLACEHOLDER, YOU WILL JUST PASTE THIS IMG IN THE DIRECTORY LOCATION)

- **LOCATION:** app/static/img/placeholder.png
- **What it does / content instructions:**
    - Generic placeholder image (your choice) for products without uploaded images. Ensure product card markup uses this when no image filenames are present.

-- FILE NAME: -- app/templates/login.html and app/templates/register.html

- **LOCATION:** app/templates/login.html, app/templates/register.html
- **What they do / content instructions:**
  - Simple pages with forms (email, password, full name for register). Form elements must have clear IDs for static/js/auth.js to read values:
    - login: id="loginEmail", id="loginPassword", id="loginSubmit".
    - register: id="regEmail", id="regName", id="regPassword", id="regConfirm", id="regSubmit".
  - Forms should not perform regular POST to server unless you want template-based flow preferable to handle via JS fetch() to JSON endpoints so the UI stays single-page-like.